MH Wilds · REFramework Object & Field Reference

DMC Combo Meter · Combat Lab Mod

All data verified against live game session · 2026-04-01

1. Verified Call Chains

All paths below were confirmed by running live Lua scripts in REFramework during a hunt session. Each arrow (->) represents a method call returning the next object.

Damage Dealt Chain

```
app.PlayerManager
  -> getMasterPlayerInfo()     -> app.cPlayerManageInfo
    -> get_Character()         -> app.HunterCharacter
      -> get_StockDamage()     -> app.cHunterStockDamage
        -> get_ApplyDamageInfo() -> app.cHunterDamageInfo
          field: _TotalDamage (f32)
          field: _IsFriendHit (bool)
```

Damage Taken Chain

```
app.HunterCharacter (from hook args[2])
  -> get_HunterHealth()     -> app.cHunterHealth
    -> get_HealthMgr()      -> app.cHealthManager
      -> get_Health()       (f32 - current HP)
      -> get_MaxHealth()    (f32 - max HP)
```

Hook Args Layout

In every REFramework sdk.hook() callback, args are indexed as follows. Calling methods on args[1] causes an immediate game crash.

Arg	Contents	Usage
`args[1]`	Thread context - raw internal pointer	Never call methods on this
`args[2]`	this pointer - the object the method was called on	sdk.to_managed_object(args[2])
`args[3]`	First method argument (if any)	sdk.to_managed_object(args[3]) if needed

2. Singleton Objects

Accessed via sdk.get_managed_singleton(name). These are global game manager objects that persist for the lifetime of a game session.

Singleton Name	Type Returned	Purpose
`app.PlayerManager`	app.PlayerManager	Entry point to all player data. Call getMasterPlayerInfo() to reach the local player info object.

3. app.cPlayerManageInfo - Methods

Returned by PlayerManager:call("getMasterPlayerInfo"). A wrapper/info object holding references to the real player objects rather than data directly.

Method	Returns	Purpose
`get_Character()`	app.HunterCharacter	Main player character object. Most combat data hangs off this.
`get_Object()`	-	Generic game object reference. Not used in combo meter.
`get_Controller()`	-	Player controller. Not used in combo meter.
`get_IsMaster()`	bool	True if this is the local player. Use to filter out network replicas in multiplayer.
`get_Valid()`	bool	Whether the player slot is currently occupied.
`get_IsCreateInfoReady()`	bool	Whether player creation data is ready. Useful as a null-safety check before accessing character data.

4. app.HunterCharacter - Key Methods

The central player object with a very large method list. Only methods relevant to the Combat Lab mod are listed. Hook targets are noted.

Method	Returns	Purpose / Notes
`get_IsMaster()`	bool	True for local player. Always check this in hooks to avoid processing network replicas.
`get_HunterHealth()`	app.cHunterHealth	Health component. Drill into this to reach the health manager and read HP values.
`get_StockDamage()`	app.cHunterStockDamage	Damage dealt data. Updated each time the hunter lands a hit. Read in evHit_AttackPostProcess post-hook.
`get_HunterStatus()`	-	Hunter status flags. Useful for future status effect tracking.
`get_HunterStamina()`	-	Stamina component. Not currently used.
`get_WeaponType()`	enum	Current weapon type. Useful for weapon-specific behaviour in future modules.
`get_CurrentActionInfo()`	-	Current animation/action state. Key for detecting specific moves in the counter tracker module.
`get_HunterMikiriInfo()`	-	Offset/mikiri counter data. Important for the counter success tracker planned for Combat Lab.
`get_DamageInfo()`	-	Incoming damage info object. Alternative path for damage taken tracking.
`evHit_AttackPostProcess()`	HOOK TARGET	Fires after each hunter attack connects. Read _TotalDamage from StockDamage in the post-hook.
`evHit_Damage()`	HOOK TARGET	Fires when the hunter receives damage. Snapshot HP in pre-hook, diff in post-hook.
`onSeamlessQuestEnd()`	HOOK TARGET	Fires on quest complete, fail, or return to hub. Used to reset combo state.

Method	Returns	Purpose / Notes
`evHit_AttackPreProcess()`	-	Fires before hit is processed. Not currently used.
`evHit_DamagePreProcess()`	-	Fires before damage is applied. Alternative to evHit_Damage pre-hook.
`get_IsCombat()`	bool	Whether the hunter is currently in combat. Useful for future idle/active HUD toggling.

5. app.cHunterStockDamage - Methods

Returned by HunterCharacter:call("get_StockDamage"). Accumulates and calculates damage data for each hit the hunter lands.

Method	Returns	Purpose
`get_ApplyDamageInfo()`	app.cHunterDamageInfo	Calculated damage info for the most recent hit. Read _TotalDamage from this.
`calcDamageValue()`	-	Internal calculation method. Do not call directly.
`calcAttrDamageValue()`	-	Elemental damage calculation. Useful for future elemental tracking.
`checkGuard()`	-	Guard check logic. Relevant to the counter/guard tracker module.
`calcGuardReaction()`	-	Calculates guard reaction type. Useful for perfect guard detection.
`calcGuardReactionTech()`	-	Tech guard (weapon-specific guard points). Key for CB guard point tracking.
`getDamageType()`	-	Returns the type of damage dealt (physical, elemental, etc).

6. app.cHunterDamageInfo - Fields

Returned by cHunterStockDamage:call("get_ApplyDamageInfo"). A native struct accessed via get_field(name). All fields verified from a live hunt session.

Field Name	Type	Example	Purpose
`_TotalDamage`	f32	7.76	Total physical damage dealt. Primary input for combo meter scoring.
`_IsFriendHit`	bool	false	True if the hit landed on a party member. Filter this out to avoid scoring friendly fire.
`_IsCounterable`	bool	true	Whether this attack can be countered by the monster. Useful for counter tracking.
`_GuardReaction`	int	0	Guard reaction level. Non-zero when the hunter blocked. Used for perfect guard detection.
`_IsSkillGuard`	bool	false	True when a skill-based guard activated (e.g. Intrepid Heart).
`_IsLackOfGuardStamina`	bool	false	True when guard broke due to insufficient stamina.
`_StunDamage`	f32	10.0	Stagger/KO damage component of this hit.
`_PoisonDamage`	f32	0.0	Poison buildup applied by this hit.

Field Name	Type	Example	Purpose
`_ParalyzeDamage`	f32	0.0	Paralysis buildup applied by this hit.
`_SleepDamage`	f32	0.0	Sleep buildup applied by this hit.
`_BlastDamage`	f32	0.0	Blast buildup applied by this hit.
`_DamageType`	int	0	Damage type enum (physical, elemental, etc).
`_IsActive`	bool	true	Whether this damage info record is active/valid.
`_IsInvalidatedDamage`	bool	false	True if damage was nullified (e.g. invincibility frame).
`_LaserHitID`	int	-1	Hit ID for laser/beam attacks. -1 means not a laser attack.
`_HitPos`	vec3	-	World position where the hit landed.
`_DamageDir`	vec3	-	Direction vector of the damage.
`_IsEnergyAttack`	bool	false	True for energy-type attacks (e.g. CB phial explosions).
`_BubbleType`	int	0	Bubble ailment type if applicable.

7. app.cHunterHealth & app.cHealthManager - Methods

app.cHunterHealth is returned by HunterCharacter:call("get_HunterHealth"). app.cHealthManager is returned by cHunterHealth:call("get_HealthMgr"). The actual HP values live on the manager, not the health component.

Object	Method	Returns	Purpose
`app.cHunterHealth`	get_HealthMgr()	app.cHealthManager	The health manager holding actual HP values.
`app.cHunterHealth`	get_IsDead()	bool	True if the hunter has died.
`app.cHunterHealth`	get_RedHealth()	-	Red (recoverable) HP component.
`app.cHunterHealth`	get_IsRedExist()	bool	Whether any red HP is present.
`app.cHunterHealth`	requestDamage()	HOOK TARGET	Alternative hook point for damage taken.
`app.cHealthManager`	get_Health()	f32	Current HP. Read before and after evHit_Damage to calculate damage taken.
`app.cHealthManager`	get_MaxHealth()	f32	Maximum HP. Used to calculate damage penalty as a proportion of max HP.
`app.cHealthManager`	get_HealthNormalized()	f32	Current HP as 0.0-1.0 fraction. Convenient for percentage-based displays.
`app.cHealthManager`	get_IsDead()	bool	True when HP reaches zero.
`app.cHealthManager`	addHealth()	-	Adds HP. Do not call in mod - for reference only.

8. REFramework Direct2D Draw API

The D2D plugin (reframework-d3d) exposes a global d2d table. The correct registration pattern, confirmed from the working Rise Switch Axe HUD mod:

```
d2d.register(init_fn, draw_fn)
  -- init_fn: called once on load. Create font objects here.
  -- draw_fn: called every frame inside the D2D context.
```

Call	Signature	Notes
`d2d.register`	d2d.register(init_fn, draw_fn)	Registers init and draw callbacks. Both required.
`d2d.Font.new`	d2d.Font.new(family, size)	Creates a font object. Call only inside init_fn.
`d2d.text`	d2d.text(font_obj, str, x, y, col)	MUST pass font object as first arg - passing an integer crashes the game.
`d2d.fill_rect`	d2d.fill_rect(x, y, w, h, col)	Draws a filled rectangle.
`d2d.image`	d2d.image(img, x, y, w, h, alpha)	Draws a loaded image.
`d2d.surface_width`	d2d.surface_width	Current render surface width in pixels.
`d2d.surface_height`	d2d.surface_height	Current render surface height in pixels.
`d2d.rect`	NOT AVAILABLE	Outline rect does not exist. Use fill_rect instead.
`d2d.on_draw`	NOT AVAILABLE	Does not exist. Use d2d.register instead.
`re.on_draw`	NOT AVAILABLE	Does not exist. Use d2d.register for drawing, re.on_frame for logic.

Colour format: 0xAARRGGBB packed integer, or helper: (a << 24) | (r << 16) | (g << 8) | b

9. Planned Future Hook Points - Combat Lab Modules

Fields and methods identified during exploration that are not yet used but are confirmed to exist and will be needed for future Combat Lab modules.

Module	Hook / Field	Object	Notes
`Counter Tracker`	get_HunterMikiriInfo()	app.HunterCharacter	Mikiri/offset data. Drill into returned object to find counter success flag.
`Counter Tracker`	_IsCounterable	app.cHunterDamageInfo	True when the incoming attack was counterable. Compare with actual counter action to calculate success rate.
`Counter Tracker`	_GuardReaction	app.cHunterDamageInfo	Non-zero on a guard. Combine with calcGuardReactionTech() to detect weapon-specific guard points.
`Counter Tracker`	calcGuardReactionTech()	app.cHunterStockDamage	Weapon-specific tech guard (guard points). Key for CB phial guard point detection.
`DPS Meter`	get_StockDamage()	app.HunterCharacter	Already hooked. Sum _TotalDamage over time and divide by elapsed seconds for DPS.

Module	Hook / Field	Object	Notes
`Time to Topple`	EnemyManager singleton	app.EnemyManager	Not yet explored. Monster topple/stagger state lives here.
`Time to Topple`	get_CurrentActionInfo()	app.HunterCharacter	Cross-reference with monster state to detect the topple moment.
`Hunter Instinct`	evHit_Damage (post-hook)	app.HunterCharacter	Zero damage result = successful dodge/iframe. Trigger slow-mo flash or heartbeat audio.
`Quest Reset`	onSeamlessQuestEnd()	app.HunterCharacter	Already hooked for combo reset. Reuse for all module resets.